C++ 数组基础

从一维数组到冒泡排序与二维数组



本专题回答四个核心问题：

1. 数组到底是什么？`sizeof(arr)` 为什么能得到整段数组所占字节？
2. 如何遍历数组、找最大值、把数组逆置，并避免越界？
3. 冒泡排序为什么“每一轮都会把一个最大元素推到右边”？
4. 二维数组在内存里到底怎样存放？行数、列数如何用 `sizeof` 计算？

适用对象：第一次系统学习 C++ 原生数组的本科初学者

2026-09-06

阅读说明

这份资料不是对手写笔记的逐字誊抄，而是按照“**为什么需要** → **直观图像** → **正式定义** → **代码** → **易错点** → **基础练习**”的顺序重新组织。原始材料中存在若干典型的初学阶段笔误或表述不严谨处，均在对应位置**显式标出并纠正**，而不是静默替换。

- 能解释 `intarr[5]` 在内存中是什么，而不是只会背语法；
- 能区分“数组总字节数”“元素个数”“单个元素字节数”；
- 能正确写出找最大值、数组逆置、冒泡排序；
- 能识别 `end=n` 导致的越界、`inttemp[n]` 在标准 C++ 中的问题；
- 能画出二维数组的“逻辑矩阵”与“行优先连续内存”两种视图；
- 能把二维成绩表按学生或按科目进行最基础的遍历与求和。

权威核验说明

本讲义关于 C++ 原生数组的连续存储、下标范围、数组边界必须是常量表达式、数组初始化、`sizeof` 语义等内容，已与 cppreference 和 Microsoft Learn 的 C++ 数组资料交叉核验；冒泡排序的基本过程与复杂度说明参考 Harvard CS50 的算法课程材料。具体来源见文末“参考资料”。[R1–R4]

目录

阅读说明 **2**

1 本专题在 C++ 学习路线中的位置 **5**

2 原始手写资料内容总览与纠错地图 **5**

3 一维数组：一排连续、同类型的数据 **6**

3.1 为什么需要数组 6
3.2 数组在内存里是什么样 7
3.3 下标：为什么第一个元素是 arr[0] 7
3.4 sizeof：字节数、元素个数、单个元素大小 7
3.5 数组名、首元素地址与 &arr：数值相近，类型不同 8
3.6 遍历：让循环负责“移动下标” 8

4 应用一：遍历数组寻找最大值 **9**

4.1 问题从哪里来 9
4.2 为什么 max = 0 不够通用 9

5 应用二：数组逆置——先理解“目标位置” 10
5.1 方法 A：借助临时数组 . . . 10
5.2 方法 B：原地双指针交换 . . . 11
5.3 两种逆置方法对比 . . . 12

6 冒泡排序：为什么最大值会一轮一轮“冒”到右边 12
6.1 它解决什么问题 . . . 12
6.2 先只看第一轮 . . . 12
6.3 为什么内层循环越来越短 . . . 13
6.4 标准基础代码 . . . 13
6.5 比较次数与时间复杂度 . . . 14

7 二维数组：先把它看成“数组的数组” 14
7.1 为什么会出现二维数组 . . . 14
7.2 逻辑矩阵与真实内存：两个视角同时保留 . . . 15
7.3 原始笔记中的四种定义/初始化方式 . . . 15
7.4 二维数组遍历：外层控制行，内层控制列 . . . 15
7.5 二维数组的 sizeof：为什么能算行数和列数 . . . 16
7.6 二维数组名与首地址 . . . 17

8 应用三：二维成绩表与按行统计 17
8.1 把现实表格映射到下标 . . . 17
8.2 输入学号，计算指定学生总分 . . . 17
8.3 另一种写法：一次求出每个学生总分 . . . 18

9 相近概念辨析：把最容易混淆的东西一次分清 19

10 从手写笔记到规范代码：完整代码复盘 19
10.1 一维数组大小与首地址 . . . 19
10.2 找最大值 . . . 19
10.3 原地逆置 . . . 20
10.4 冒泡排序 . . . 20
10.5 二维数组行列统计 . . . 20

11 知识地图：把本专题串成一条线 21

12 学习自检：不看答案先口头回答 21

13 正式基础练习（共 24 题） 22
13.1 单项选择题（10 题） . . . 22

13.2 多项选择题（5 题） 23
13.3 填空题（5 题） 24
13.4 基础解答题（4 题） 24

14 答案速查 **25**
14.1 单项选择 25
14.2 多项选择 25
14.3 填空 25
14.4 解答题结果 25

15 单项选择题详细解析 **26**

16 多项选择题详细解析 **27**

17 填空题详细解析 **28**

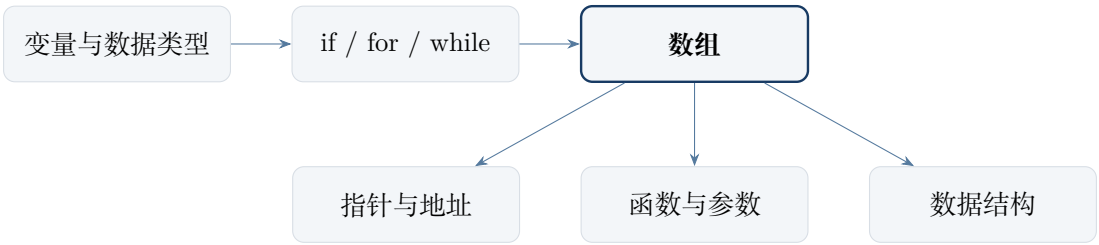
18 基础解答题详细解析 **28**

19 高频易错点总表 **30**

20 现代 C++ 的下一步：知道即可，不要求现在掌握 **30**

21 参考资料 **30**

1 本专题在 C++ 学习路线中的位置



数组是你从“单个变量”走向“批量数据”的第一座桥。后面学习指针时，`arr` 与 `&arr[0]` 的关系会再次出现；学习数据结构时，顺序表、栈、队列等结构都离不开“连续存储 + 下标访问”的思想。因此，本专题不只是语法点，而是后续编程课的基础坐标。

> **普通变量像一个抽屉，数组像一排编号连续、规格相同的抽屉**。你用数组名找到这排抽屉，用下标找到其中第几个抽屉。

2 原始手写资料内容总览与纠错地图

根据上传的 10 张课堂手写笔记，本专题覆盖：一维数组的用途、`sizeof` 与数组首地址、找最大值、数组逆置、冒泡排序、二维数组的四种定义/初始化方式、二维数组的行列统计，以及二维成绩表求总分。

模块	原始资料核心内容	本讲义处理
一维数组	`sizeof(arr)`、元素个数、数组首地址	规范术语 + 内存图
最大值	遍历 5 个猪的体重寻找最大值	补充负数边界条件
逆置	临时数组法、首尾交换法	修复 VLA 与越界/漏赋值
冒泡排序	相邻比较、交换、双重循环	补充每轮不变量与复杂度图
二维数组	4 种定义/初始化、嵌套遍历	加入行优先内存图
成绩统计	二维成绩表按学号求总分、逐行求和	纠正数据不一致并加输入检查

1. “**数组名是常量**”是便于初学的说法，但不够规范；更准确地说：**数组对象不能整体赋值，数组名在多数表达式中会转换为指向首元素的指针**。
2. `sizeof(arr)` 得到的是**整段数组占用的字节数**，不是“元素个数”；元素个数要再除以 `sizeof(arr[0])`。

3. 找最大值时写 `intmax=0;` 只对“所有元素非负”安全；通用写法应从 `arr[0]` 开始。
4. 临时数组逆置中出现 `inttemp_arr[n];`。当 `n` 是运行期普通变量时，**这不是标准 C++ 的变长数组语法**。
5. 原地逆置的手写代码把 `end` 设成了元素个数，并漏掉一次对称赋值，存在**越界与逻辑错误**；应使用 `end=n-1` 且完整交换两端元素。
6. 成绩案例的表格数据与后续代码初始化数据存在不一致。本讲义把这种情况作为“**原始资料内部不一致**”明确保留说明，并统一选用表格所示数据演示算法。

无法可靠逐字辨认的内容

部分图片中的中文 `cout` 提示字符串被修正带遮挡或字迹重叠。由于这些字符串只承担输出文案作用，不影响 C++ 语法与算法逻辑，本讲义不强行猜测其原句，而统一改写为清晰、等价的教学提示语。

3 一维数组：一排连续、同类型的数据

3.1 为什么需要数组

如果要保存 5 个体重，当然可以写：

```
int w1 = 300;
int w2 = 350;
int w3 = 200;
int w4 = 400;
int w5 = 250;
```

但一旦需要“把每个体重都检查一遍”，你就会发现单独变量很难配合循环。数组把同一类数据统一装进一个有编号的容器中，于是可以用一个下标 `i` 依次访问。

定义：一维原生数组（built-in array）

写作

`T arr[N];`

表示：创建一个名为 `arr` 的数组对象，其中包含 N 个**类型相同**的元素，每个元素类型为 `T`。C++ 原生数组的元素连续存储，下标从 0 到 $N-1$。[R1,R2]

最基础例子

```
int arr[5] = {1, 2, 3, 4, 5};
```

这里：数组名是 `arr`，元素类型是 `int`，元素个数是 5，有效下标是 0、1、2、3、4。

3.2 数组在内存里是什么样

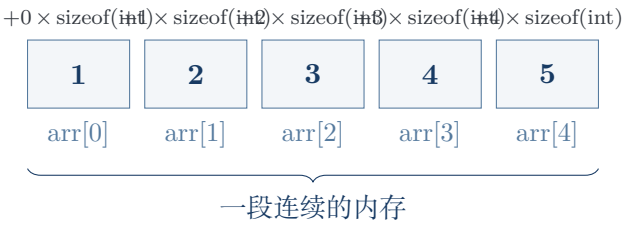


图 1: 一维数组的逻辑下标与连续内存布局

图中最重要的不是具体地址，而是**相邻元素在内存中相邻**。如果一个 `int` 占 4 字节，那么相邻两个元素的起始地址通常相差 4 字节；真正的大小应以当前平台上的 `sizeof(int)` 为准。

3.3 下标：为什么第一个元素是 arr[0]

数组下标是“相对首元素的偏移量”。`arr[0]` 表示从数组开头偏移 0 个元素；`arr[1]` 表示偏移 1 个元素。因此长度为 N 的数组最后一个元素必然是 `arr[N-1]`。

对 `intarr[5]`，`arr[5]` 已经是第 6 个位置，**越界**。C++ 原生数组不会自动替你检查下标是否合法，越界访问会产生未定义行为。

3.4 sizeof：字节数、元素个数、单个元素大小

三个最容易混淆的量

对于当前作用域内真实存在的数组对象 `intarr[5]`：

$$\boxed{\text{总字节数} = \texttt{sizeof(arr)}}$$

$$\boxed{\text{单个元素字节数} = \texttt{sizeof(arr[0])}}$$

$$\boxed{\text{元素个数} = \frac{\texttt{sizeof(arr)}}{\texttt{sizeof(arr[0])}}}$$

`sizeof` 的结果类型是 `std::size_t`，含义是对象所占存储大小（以字节计）。[R3]

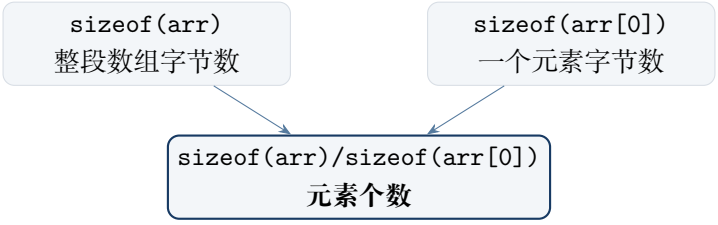


图 2: 用 `sizeof` 从“字节”得到“元素个数”

```
1 #include <iostream>
2 using namespace std;
3
```

```cpp
int main() {
    int arr[] = {1, 2, 3, 4, 5};

    cout << "array bytes = " << sizeof(arr) << '\n';
    cout << "element count = " << sizeof(arr) / sizeof(arr[0]) << '\n';
    cout << "first-element address = " << arr << '\n';
    return 0;
}
```

原图思路：通过 sizeof(arr) 统计整个数组“在内存中的长度”。
更规范：最好说“数组总字节数”。如果说“数组长度”，通常更容易让人理解成元素个数 5。两者必须分开。

3.5 数组名、首元素地址与 &arr：数值相近，类型不同

手写资料写“数组名可以获取数组首地址”，这对初学阶段方向是对的，但需要补上一个重要细节：数组名 arr 在**多数表达式**中会发生数组到指针转换，得到指向首元素的指针，即近似理解为 &arr[0]。[R1]

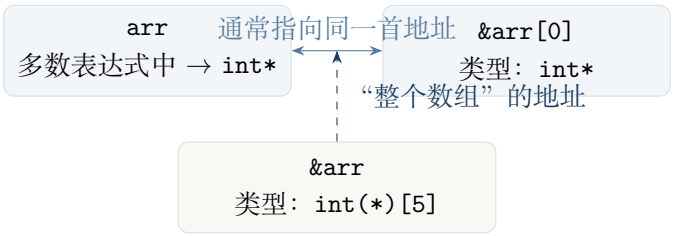


图 3: arr、&arr[0]、&arr 的关系

原图写法：数组名是常量，不可以进行赋值操作。
存在的问题：“常量”容易让人误以为数组元素也不能改。实际上 arr[0]=100; 完全可以。
规范理解：数组对象不能整体通过赋值运算符重新绑定或拷贝，例如 arr=other; 不合法；数组名在多数表达式中会转换为首元素指针，但它本身不是一个可以像普通指针变量那样重新赋值的对象。

3.6 遍历：让循环负责“移动下标”

数组遍历

如果数组有 n 个元素，最经典的遍历形式是

```cpp
for (int i = 0; i < n; ++i) {
    // use arr[i]
}
```

下标从 0 开始；循环条件使用 i<n，因此最后一次访问恰好是 arr[n-1]。

自检：这三个条件谁会越界？

对 `intarr[5]`：

- `i<5`：✓ 安全范围 0–4；
- `i<=4`：✓ 同样访问 0–4；
- `i<=5`：✗ 会访问 `arr[5]`。

4 应用一：遍历数组寻找最大值

4.1 问题从哪里来

原始资料给出 5 只小猪的体重，要求找出最重的一只。抽象以后，问题就是：**给定一组数，逐个检查，保留“目前见过的最大值”**。

每读到一个新元素，只需要问一句：**它比当前最大值更大吗？**如果更大，就让它成为新的最大值；否则最大值保持不变。

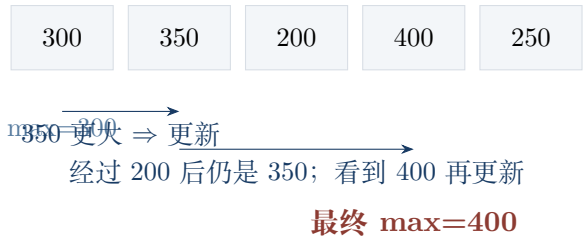


图 4: 最大值扫描：最大值只在“遇到更大元素”时更新

4.2 为什么 max = 0 不够通用

原始笔记使用 `intmax=0;`。对体重这样的正数数据当然能得到正确结果；但如果数组是 `\{-5,-2,-9\}`，0 并不在数组里，却会一直保持“最大值”，答案就错了。

通用初始化原则

如果数组非空，最稳妥的基础写法是

```
int maxValue = arr[0];
```

然后从下标 1 开始比较。这样“当前最大值”始终来自数组本身。

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {300, 350, 200, 400, 250};
    int maxValue = arr[0];

```

```cpp
    for (int i = 1; i < 5; ++i) {
        if (arr[i] > maxValue) {
            maxValue = arr[i];
        }
    }

    cout << "max = " << maxValue << '\n';
    return 0;
}
```

你正在学习一种非常重要的程序设计模式：**扫描（scan）+ 状态变量（state）**。后面求最小值、求和、计数、统计满足条件的元素，本质上都是“遍历 + 按规则更新状态”。

5 应用二：数组逆置——先理解“目标位置”

给定

$$[1, 2, 3, 4, 5]$$

逆置后要得到

$$[5, 4, 3, 2, 1].$$

“逆置”不是排序，因为数值大小不重要；它只改变元素的位置顺序。

5.1 方法 A：借助临时数组

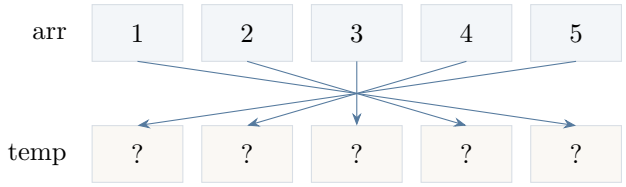


图 5: 临时数组法：原下标 i 映射到新下标 $n-1-i$

最直接的下标关系是

```
temp[i] = arr[n - 1 - i]
```

或等价地从末尾读取原数组。

```cpp
#include <iostream>
using namespace std;

int main() {
    constexpr int n = 5;
    int arr[n] = {1, 2, 3, 4, 5};
    int temp[n];

    for (int i = 0; i < n; ++i) {
        temp[i] = arr[n - 1 - i];
    }
```

```
12
13      for (int i = 0; i < n; ++i) {
14          cout << temp[i] << ' ';
15      }
16      cout << '\n';
17      return 0;
18  }
```

原图先用 `intn=sizeof(arr)/sizeof(arr[0]);` 得到长度，再写 `inttemp_arr[n];`。GCC 可能把它作为扩展接受，但**标准 C++ 的内置数组边界需要常量表达式**；Microsoft C++ 文档也明确说明栈上原生数组的元素个数需要编译期常量。[R1,R2]
基础阶段可以直接使用 `constexprintn=5;`；如果长度真的要到运行时才知道，应改用 `std::vector`。

5.2 方法 B：原地双指针交换

更节省额外空间的方法是：让 `start` 从左端向右走，让 `end` 从右端向左走，每次交换一对对称元素。



图 6: 原地逆置的第一步：交换最左与最右元素

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int n = sizeof(arr) / sizeof(arr[0]);

    int start = 0;
    int end = n - 1;

    while (start < end) {
        int temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;
        ++start;
        --end;
    }

    for (int x : arr) {
        cout << x << ' ';
    }
```

```
22      cout << '\n';
23      return 0;
24  }
```

错误 1： 原图把 end 初始化为元素个数 n。如果 n=5，那么 arr[5] 越界。正确最后下标是 n-1=4。
错误 2： 原图交换区间中可见写法相当于“保存 arr[start]，然后给 arr[end] 两次赋值”，漏掉了 arr[start]=arr[end];。完整三步交换不能少任何一步。
推荐循环条件： start<end。当二者相遇时，中间元素无需再与自己交换。

5.3 两种逆置方法对比

项目	临时数组法	原地双指针法
核心思路	把元素按反向顺序写入新数组	交换左右对称元素
额外空间	需要一个同规模数组	只需一个临时变量
下标难点	n-1-i	start/end 边界
初学优势	直观、映射关系清楚	能训练边界与交换思维

6 冒泡排序：为什么最大值会一轮一轮“冒”到右边

6.1 它解决什么问题

逆置只关心位置，排序则要求元素满足大小关系。冒泡排序的基本操作非常简单：**每次只比较相邻两个元素，如果顺序反了就交换**。

冒泡排序的核心规则

升序排序时，对相邻元素 arr[j] 与 arr[j+1]：

$$\boxed{\text{若} arr[j] > arr[j+1], \text{交换二者}}$$

从左到右做完一轮后，当前未排序区间中的最大元素会被连续交换到最右端。[R4]

6.2 先只看第一轮

以原始资料中的数组

$$[4, 7, 6, 8, 5, 2, 1, 3, 9]$$

为例。为了让“冒泡”动作看得更明显，下面另外用短数组 $[5, 1, 4, 2, 3]$ 展示第一轮：

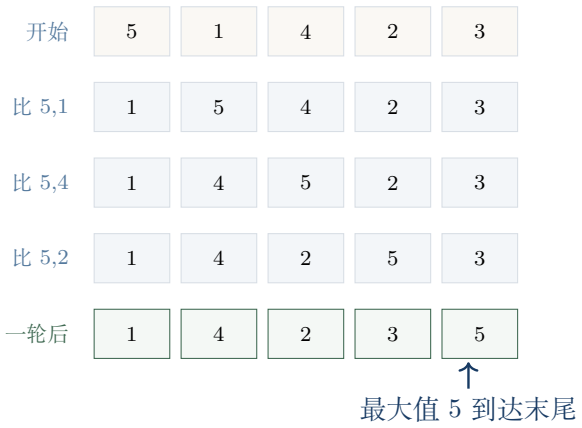


图 7: 冒泡排序第一轮：大元素不断与右邻居交换，因此向右移动

6.3 为什么内层循环越来越短

第一轮结束后，最后一个元素已经确定；第二轮不必再比较它。第二轮后，最后两个元素确定。于是第 i 轮（从 $i=0$ 计）只需要比较

$$\boxed{n-i-1}$$

对相邻元素。

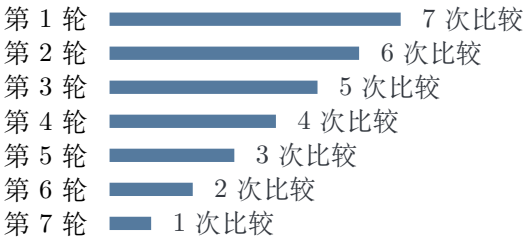


图 8: 示意：随着右侧有序区增长，每轮比较次数递减

6.4 标准基础代码

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[] = {4, 7, 6, 8, 5, 2, 1, 3, 9};
    int n = sizeof(arr) / sizeof(arr[0]);

    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - i - 1; ++j) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    for (int x : arr) {
```

```
        cout << x << ' ';
    }
    cout << '\n';
    return 0;
}
```

外层第 i 轮结束后，数组最右侧已经有 $i+1$ 个元素处于最终正确位置。这个“**右侧有序区逐轮扩大**”就是你为什么能写出 `j<n-i-1` 的根本原因，而不是死记循环上界。

6.5 比较次数与时间复杂度

不做“提前退出”优化时，比较次数是

$$(n-1)+(n-2)+\cdots+1=\frac{n(n-1)}{2},$$

数量级是 $O(n^2)$。Harvard CS50 也以重复相邻交换的方式介绍冒泡排序，并给出最坏情形 $O(n^2)$；若加入“本轮无交换则提前结束”的标志，最好情况可在一次扫描后结束。[R4]

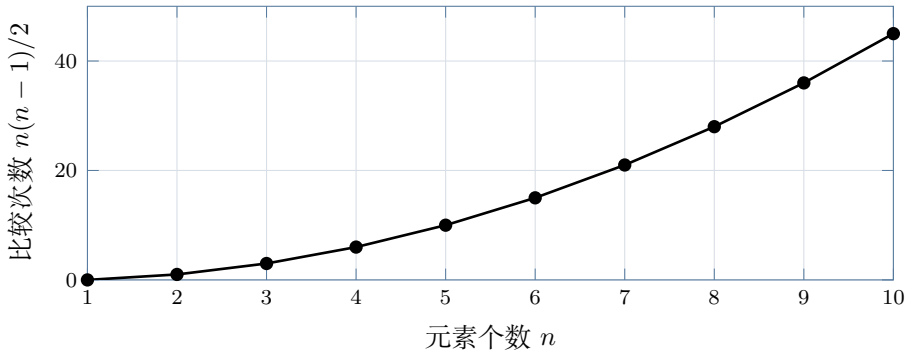


图 9: 冒泡排序比较次数随 n 近似二次增长

拓展联想：为什么课程还要学冒泡排序

冒泡排序并不是现代 C++ 中实际排序的首选；但它非常适合初学者同时练习**数组下标、双重循环、边界、交换、循环不变量和复杂度**。真正工程代码通常直接使用标准库排序算法，而不是手写冒泡。

7 二维数组：先把它看成“数组的数组”

7.1 为什么会出现二维数组

当数据天然带有“行”和“列”两层编号时，一维数组就不够直观。例如：3 名学生，每人 3 门课；图像中的像素矩阵；棋盘；表格。二维数组让我们写成 `score[ ][ ]`。

定义：二维原生数组

```
T arr[ROWS][COLS];
```

它不是一个神秘的新结构，而是：**一个长度为 ROWS 的数组，其中每个元素又是一个长度为 COLS 的数组**。因此 `arr[i]` 表示第 i 行，`arr[i][j]` 表示第 i 行第 j 列元素。[R1,R2]

7.2 逻辑矩阵与真实内存：两个视角同时保留



图 10: 二维数组在 C++ 中按“行数组”顺序连续存储

7.3 原始笔记中的四种定义/初始化方式

设目标是一个 2 行 3 列整型数组：

四种常见写法

```cpp
// 1. Only declare: values are not initialized automatically for local array
int arr1[2][3];

// 2. Nested initializer lists
int arr2[2][3] = {{1, 2, 3}, {4, 5, 6}};

// 3. Flat initializer list
int arr3[2][3] = {1, 2, 3, 4, 5, 6};

// 4. Omit the first dimension when initializer is present
int arr4[][3] = {1, 2, 3, 4, 5, 6};
```

二维数组的元素类型其实是“`int[3]`”。编译器必须知道**每一行有几个元素**，才能根据 `arr[i][j]` 计算从首地址偏移多少。因此在这种初始化写法里，可以让编译器通过初始化元素推断“有几行”，但列数必须已知。cppreference 对多维数组的未知边界也明确指出，除第一维外不能留下未知边界。[R1]

7.4 二维数组遍历：外层控制行，内层控制列

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[2][3] = {{1, 2, 3}, {4, 5, 6}};

```

```
    for (int i = 0; i < 2; ++i) {
        for (int j = 0; j < 3; ++j) {
            cout << "arr[" << i << "][" << j << "] = "
                << arr[i][j] << '\n';
        }
    }
    return 0;
}
```

这里只是约定俗成：i 常表示行、j 常表示列。真正决定含义的是你把谁写进 arr[\protect\protect\leavevmode@ifvmode\kern+.1667em\relax][\protect\protect\leavevmode@ifvmode\kern+.1667em\relax] 的第一个和第二个下标位置，以及循环边界分别是多少。

7.5 二维数组的 sizeof：为什么能算行数和列数

对

```
int arr[2][3] = {{1,2,3}, {4,5,6}};
```

有三层大小：

- sizeof(arr)：整个二维数组的总字节数；
- sizeof(arr[0])：第 0 行整个一维数组的字节数；
- sizeof(arr[0][0])：一个整数元素的字节数。

因此

行数与列数

$$\boxed{\text{行数} = \frac{\texttt{sizeof(arr)}}{\texttt{sizeof(arr[0])}}}$$

$$\boxed{\text{列数} = \frac{\texttt{sizeof(arr[0])}}{\texttt{sizeof(arr[0][0])}}}$$

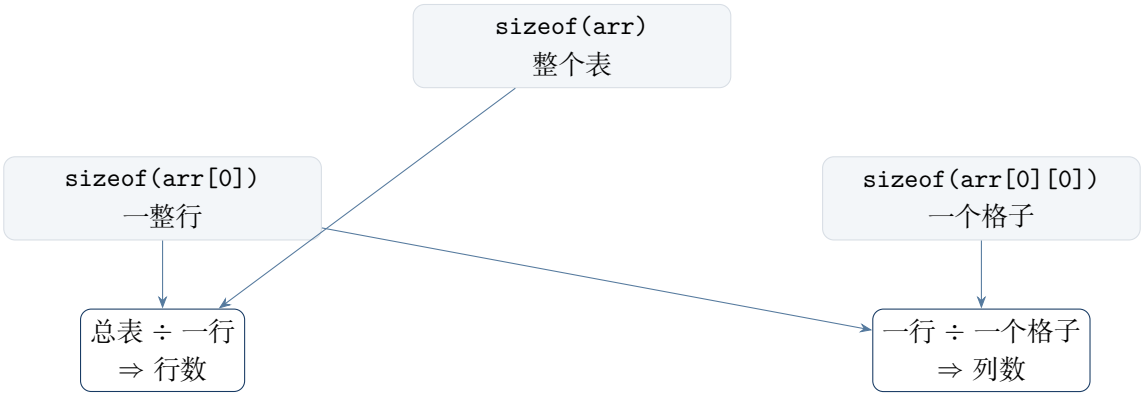


图 11: 把二维数组理解成“总表 ÷ 行”“一行 ÷ 单元格”

上述公式成立的关键是：arr 在当前作用域里仍然是一个真实的数组对象。若把数组作为普通函数参数传入，数组参数会调整为指针，此时 sizeof(arr) 得到的是指针大小，不再是原数组总大小。这是后面学习“数组与函数”时必须再次注意的点。[R1,R3]

7.6 二维数组名与首地址

对 intarr[2][3]，表达式 arr 在多数情况下转换为“指向第 0 行的指针”，其类型可理解为 int(*)[3]；arr[0] 又会转换为指向第 0 行第 0 个元素的 int*。这解释了为什么二维数组做指针运算时“跨一行”和“跨一个元素”是两种不同步长。

8 应用三：二维成绩表与按行统计

8.1 把现实表格映射到下标

原始资料使用“学生 × 科目”的成绩表。根据图片中表格可辨认的一组数据，本讲义统一采用：

学生	语文	数学	英语
张三（学号 1）	100	100	100
李四（学号 2）	90	50	100
王五（学号 3）	60	70	80

于是可写成：

```
int score[3][3] = {
    {100, 100, 100},
    { 90, 50, 100},
    { 60, 70, 80}
};
```

后续手写代码页中可辨认的初始化数据与前一页表格并不一致。由于这不会改变“二维数组按行求和”的算法，本讲义不把其中某一组悄悄当成唯一正确答案，而是明确说明：**数据内容不一致，算法结构一致**。为了让表格、代码、计算结果统一，正文采用上表数据。

8.2 输入学号，计算指定学生总分

如果学号 1、2、3 分别对应数组行 0、1、2，那么数组行下标就是

```
studentIndex = studentId - 1
```

```
#include <iostream>
using namespace std;

int main() {
    int score[3][3] = {
```

```
        {100, 100, 100},
        { 90, 50, 100},
        { 60, 70, 80}
    };

    int studentId;
    cout << "student id (1-3): ";
    cin >> studentId;

    if (studentId < 1 || studentId > 3) {
        cout << "invalid id\n";
        return 0;
    }

    int total = 0;
    for (int j = 0; j < 3; ++j) {
        total += score[studentId - 1][j];
    }

    cout << "total = " << total << '\n';
    return 0;
}
```

原始手写代码直接使用 score[student_id-1][i]。如果用户输入 0 或 4，就会得到负下标或越界下标。数组本身不会拦住这种错误，因此程序需要在索引前主动验证范围。

8.3 另一种写法：一次求出每个学生总分

```
for (int i = 0; i < 3; ++i) {
    int sum = 0;
    for (int j = 0; j < 3; ++j) {
        sum += score[i][j];
    }
    cout << "student " << (i + 1) << " total = " << sum << '\n';
}
```

外层 i 固定一名学生，内层 j 把这一行的三门课依次累加。于是这段代码本质上是“**逐行归约 (row-wise reduction)**”。



图 12: 固定行下标，对列下标求和

9 相近概念辨析：把最容易混淆的东西一次分清

概念	正确理解	最易犯错
数组总字节数	`sizeof(arr)`	把它直接当元素个数
元素个数	`sizeof(arr)/sizeof(arr[0])`	在函数参数中仍照搬该公式
最后下标	$n-1$	写成 n，导致越界
逆置	改变元素位置顺序	误以为等同于从大到小排序
冒泡排序	相邻比较，顺序错误则交换	以为每次直接“找到最大值并放过去”
arr	多数表达式中转为首元素指针	误把“数组名不能赋值”理解成元素不能改
二维数组	“数组的数组”	只把它想象成数学矩阵，不理解内存布局
arr[0]	第 0 行这个一维数组	误以为它只是一个整数

10 从手写笔记到规范代码：完整代码复盘

10.1 一维数组大小与首地址

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    const int n = sizeof(arr) / sizeof(arr[0]);

    cout << "bytes: " << sizeof(arr) << '\n';
    cout << "count: " << n << '\n';
    cout << "first address: " << arr << '\n';
    return 0;
}
```

10.2 找最大值

```cpp
int maxValue = arr[0];
for (int i = 1; i < n; ++i) {
    if (arr[i] > maxValue) {
        maxValue = arr[i];
    }
}
```

10.3 原地逆置

```
int start = 0;
int end = n - 1;
while (start < end) {
    int temp = arr[start];
    arr[start] = arr[end];
    arr[end] = temp;
    ++start;
    --end;
}
```

10.4 冒泡排序

```
for (int i = 0; i < n - 1; ++i) {
    for (int j = 0; j < n - i - 1; ++j) {
        if (arr[j] > arr[j + 1]) {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}
```

10.5 二维数组行列统计

```
int rows = sizeof(arr) / sizeof(arr[0]);
int cols = sizeof(arr[0]) / sizeof(arr[0][0]);
```

写数组题时，先检查 5 件事：

1. 有效下标到底是 0 到多少？
2. 循环条件有没有把 n 当成合法下标？
3. 初始状态是否来自数组本身（例如最大值）？
4. `sizeof` 此时面对的是“真实数组”还是“已经退化/调整成指针的参数”？
5. 二维数组的外层、内层循环上界分别对应行还是列？

11 知识地图：把本专题串成一条线

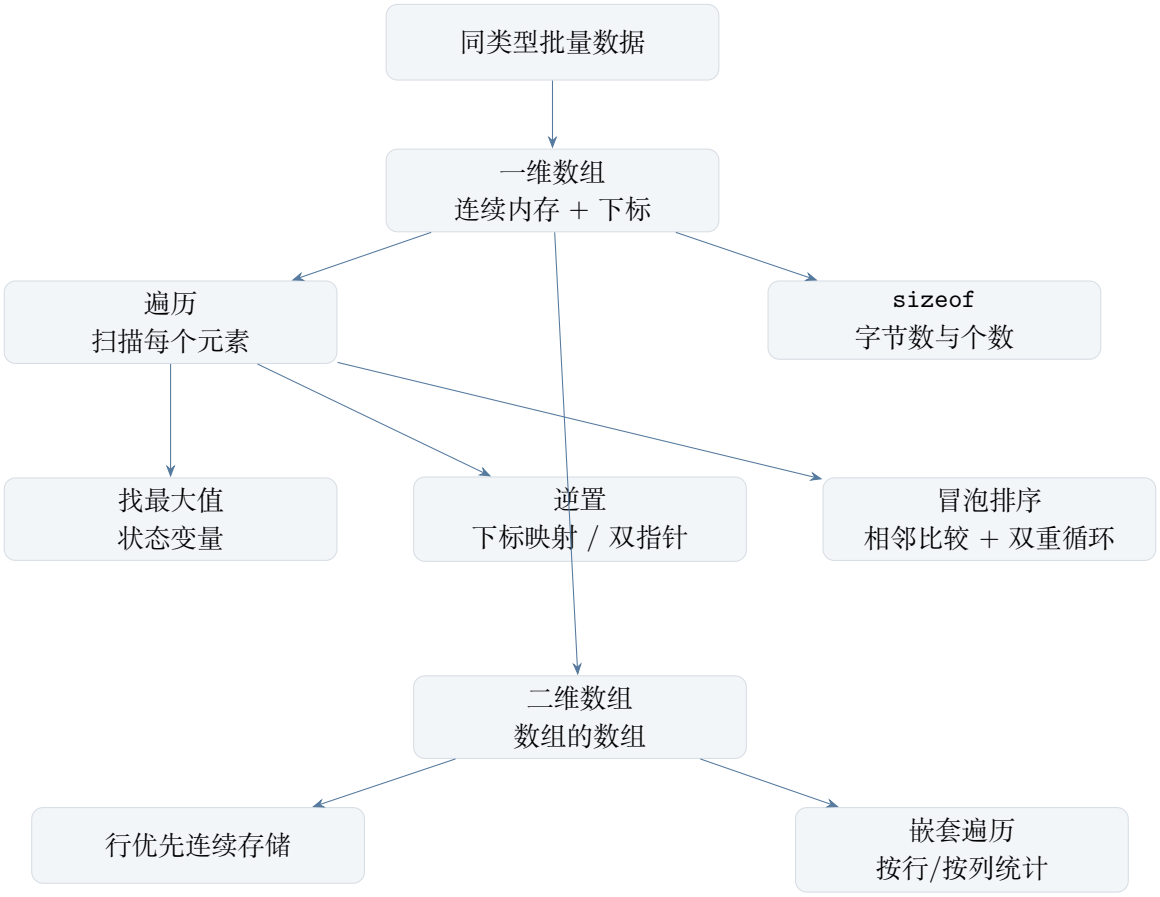


图 13: 本专题知识关系图

后续课程连接

指针： 数组名与首地址、指针算术； **数据结构：** 顺序表与连续存储； **算法：** 排序、查找与复杂度； **机器人/工程编程：** 传感器采样序列、关节状态向量、图像/栅格地图都可以从“批量同类型数据 + 索引”这一思想继续发展。

12 学习自检：不看答案先口头回答

以下问题不计入正式 24 道练习。

1. `intarr[5]` 的有效下标为什么是 0–4?
2. `sizeof(arr)` 与元素个数有什么区别?
3. 为什么找最大值用 `arr[0]` 初始化比用 0 更通用?
4. 原地逆置为什么要让 `end=n-1`?
5. 冒泡排序一轮结束后，哪个位置一定更接近最终状态?
6. 为什么冒泡排序的内层比较次数会逐轮减少?
7. `intarr[2][3]` 从“数组的数组”角度怎样解释?
8. 二维数组行数与列数分别如何用 `sizeof` 计算?

9. 为什么在函数参数中不能总用 `sizeof(arr)/sizeof(arr[0])` 求原数组长度?

10. 成绩表中，如果学号从 1 开始，而数组下标从 0 开始，两者如何转换?

13 正式基础练习（共 24 题）

题型	单项选择	多项选择	填空	基础解答
数量	10	5	5	4

13.1 单项选择题（10 题）

1. 对 `intarr[5]=\{1,2,3,4,5\};`，最后一个元素是（　）。

A. `arr[5]`

B. `arr[4]`

C. `arr[0]`

D. `arr[-1]`

2. 若当前作用域中有 `intarr[5];`，计算元素个数的基础表达式是（　）。

A. `sizeof(arr)`

B. `sizeof(arr[0])`

C. `sizeof(arr)/sizeof(arr[0])`

D. `sizeof(arr[0])/sizeof(arr)`

3. 对非空整型数组寻找最大值时，最通用的基础初始化是（　）。

A. `intmaxValue=0;`

B. `intmaxValue=arr[0];`

C. `intmaxValue=-1;`

D. `intmaxValue=arr[n];`

4. 长度为 n 的数组，原地逆置时右端初始下标应为（　）。

A. n

B. $n-1$

C. 1

D. 0

5. 升序冒泡排序中，如果 `arr[j]>arr[j+1]`，应该（　）。

A. 删除两元素

B. 交换两元素

C. 跳出所有循环

D. 把二者都置 0

6. 对 n 个元素执行未优化的基础冒泡排序，外层循环通常需要（　）轮。

A. 1

B. n

C. $n-1$

D. $n+1$

7. 对 `inta[2][3];`，`a[1][2]` 表示（　）。

A. 第 1 行第 2 列（按 1 起始口语编号）

B. 下标为 1 的行、下标为 2 的列

C. 数组总大小

D. 第 2 行首地址

8. 对 `inta[2][3];`，行数的基础计算式是（　）。

A. `sizeof(a)/sizeof(a[0])`

B. `sizeof(a[0])/sizeof(a[0][0])`

C. `sizeof(a)`

D. `sizeof(a[0][0])`

9. 下列关于 `inta[][3]=\{1,2,3,4,5,6\};` 的说法正确的是（　）。

A. 第一维由初始化元素推断为 2

B. 列数由编译器任意决定

C. 数组只有 3 个元素

D. 这种写法在 C++ 中一定非法

10. 若成绩数组 `score[3][3]` 的第 0 行表示学号 1，则学号 `id` 对应的行下标应为（　）。

A. `id`

B. `id+1`

C. `id-1`

D. `3-id`

13.2 多项选择题（5 题）

1. 关于 `intarr[5]=\{1,2,3,4,5\};`，正确的有（　）。

A. 元素连续存储

B. 有效下标是 0–4

C. `arr[5]` 是最后一个元素

D. `arr[0]` 的类型是 `int`

2. 关于 `sizeof`，正确的有（　）。

A. 对当前作用域的真实数组，`sizeof(arr)` 给出整个数组字节数

B. `sizeof(arr[0])` 给出一个元素字节数

C. 元素个数可用二者相除得到

D. 在任何函数参数里都能用同样公式得到调用者原数组长度

3. 对原地逆置算法，正确的有（　）。

A. `start` 初始可设为 0

B. `end` 初始应设为 `n-1`

C. 每轮交换 `arr[start]` 与 `arr[end]`

D. 每轮结束后 `start--`、`end++`

4. 关于升序冒泡排序，正确的有（　）。

A. 比较相邻元素

B. 若左元素更大则交换

C. 基础版本最坏时间复杂度为 $O(n^2)$

D. 第一轮结束后最小元素一定在最右端

5. 关于二维数组 `inta[2][3]`，正确的有（　）。

A. 可以理解为 2 个“长度为 3 的 int 数组”组成的数组

B. `a[0]` 表示第 0 行这个一维数组

C. `a[0][0]` 表示一个 int 元素

D. 列数可由 `sizeof(a)/sizeof(a[0])` 得到

13.3 填空题（5 题）

1. `intarr[8];` 的最后一个合法下标是 _____。

2. 对真实数组 `intarr[6];`，元素个数可写为 _____。

3. 原地逆置中，若 `intn=5;`，则 `end` 初始值应为 _____。

4. 冒泡排序外层下标为 `i`（从 0 开始）时，内层常用上界是 `j<` _____。

5. 对 `inta[4][7];`，列数公式为 _____。

13.4 基础解答题（4 题）

1. 已知 `intarr[5]=\{12,7,19,3,10\};`，写出寻找最大值的完整核心代码，并说明为什么不建议把最大值初始设为 0。

2. 已知 `intarr[5]=\{1,2,3,4,5\};`，使用首尾交换法将其原地逆置。要求写出 `start`、`end` 的初值、循环条件和交换代码。

3. 对数组 `intarr[5]=\{5,1,4,2,3\};`，手工写出升序冒泡排序**第一轮**结束后的数组，并解释为什么本轮后最右端元素已经确定。

4. 已知

```cpp
int score[3][3] = {
    {100, 100, 100},
    { 90, 50, 100},
    { 60, 70, 80}
};
```

写出计算每一名学生总分的双重循环，并给出三名学生的总分。

14 答案速查

14.1 单项选择

1	2	3	4	5	6	7	8	9	10
B	C	B	B	B	C	B	A	A	C

14.2 多项选择

1	2	3	4	5
ABD	ABC	ABC	ABC	ABC

14.3 填空

1. 7
2. `sizeof(arr)/sizeof(arr[0])`
3. 4
4. `n-i-1`
5. `sizeof(a[0])/sizeof(a[0][0])`

14.4 解答题结果

1. 最大值 19；推荐从 `arr[0]` 初始化。
2. 逆置结果 `54321`。
3. 第一轮后 `14235`。
4. 三人总分依次为 300、240、210。

15 单项选择题详细解析

1. **正确答案：B。**

   考查：下标范围。长度为 5 的数组有效下标从 0 到 4，因此最后元素是 `arr[4]`。A 的 `arr[5]` 越界；C 是第一个元素；D 使用负下标同样越界。

   一句话：长度 n 的最后下标永远是 $n-1$。

2. **正确答案：C。**

   考查：`sizeof` 与元素个数。`sizeof(arr)` 是总字节数，`sizeof(arr[0])` 是单个元素字节数，相除才得到元素个数。A、B 都是字节数；D 把除法方向写反。

   易错：不要把“字节数”直接叫“数组长度”。

3. **正确答案：B。**

   考查：最大值初始化。用 `arr[0]` 能保证初始最大值确实来自数组。A 对全负数组会失败；C 同理只在部分数据可用；D 的 `arr[n]` 越界。

   一句话：扫描极值时，让初始候选来自数据本身。

4. **正确答案：B。**

   考查：最后合法下标。长度为 n 时下标为 0 到 $n-1$，因此右端必须从 $n-1$ 开始。A 会越界；C、D 不是一般的右端位置。

   易错：“元素个数”和“最后下标”差 1。

5. **正确答案：B。**

   考查：冒泡排序的相邻交换规则。升序时左边更大说明顺序反了，应交换。其余选项都不属于冒泡排序的基本动作。

   一句话：冒泡排序每次只纠正一对相邻元素的局部次序。

6. **正确答案：C。**

   考查：基础冒泡排序轮数。每轮至少把一个未排序区间的最大元素放到右端；最多需要 $n-1$ 轮。

   易错：外层轮数与内层比较次数不是同一个量。

7. **正确答案：B。**

   考查：二维下标含义。C++ 下标从 0 开始，所以 `a[1][2]` 是“行下标 1、列下标 2”的元素，也就是按人类从 1 开始数时第 2 行第 3 列。A 的口语编号不准确；C、D 不是元素访问。

   一句话：第一个下标选行，第二个下标选行内元素。

8. **正确答案：A。**

   考查：二维数组行数。整个数组总字节数除以一行字节数得到行数。B 得到列数；C、D 只是字节数。

   一句话：总表 ÷ 一行 = 行数。

9. **正确答案：A。**

   考查：二维数组初始化与省略第一维。每行 3 个元素，共有 6 个初始化值，因此第一维推断为 2。B 错在列数不能任意；C 忽略了第二行；D 与标准语法不符。

   一句话：初始化时可以让编译器数“有几行”，但必须知道“一行多宽”。

10. **正确答案：C。**

考查： 现实编号到数组下标的映射。学号从 1 开始、数组从 0 开始，因此减 1。

易错： 使用前还应验证 `id` 是否在 1–3。

16 多项选择题详细解析

1. **正确答案：A、B、D。**

A：✓ 原生数组元素连续存储。

B：✓ 5 个元素下标为 0–4。

C：✗ `arr[5]` 是越界位置，不是最后元素。

D：✓ `arr[0]` 是一个 `int` 元素。

总结： 数组整体与单个元素要分清。

2. **正确答案：A、B、C。**

A：✓ 对当前作用域内真实数组，`sizeof(arr)` 是整段字节数。

B：✓ `sizeof(arr[0])` 是单个元素大小。

C：✓ 相除得到元素个数。

D：✗ 普通函数参数中的数组会调整为指针，`sizeof` 不再保留调用者原数组总大小。

总结： 公式有效的前提是“这里仍然是数组对象”。

3. **正确答案：A、B、C。**

A：✓ 左端从 0 开始。

B：✓ 右端最后合法下标为 $n-1$。

C：✓ 每轮交换对称位置。

D：✗ 应是 `start++`、`end--`，让两端向中间靠拢。

总结： 双指针逆置的方向必须“向内收缩”。

4. **正确答案：A、B、C。**

A：✓ 冒泡排序比较相邻元素。

B：✓ 升序时左大右小就交换。

C：✓ 基础版本最坏为 $O(n^2)$。

D：✗ 第一轮后到最右端的是当前最大元素，不是最小元素。

总结： “大元素向右冒”对应升序版本。

5. **正确答案：A、B、C。**

A：✓ 二维数组就是数组的数组。

B：✓ `a[0]` 是第一行这个长度为 3 的数组。

C：✓ 再取一个下标才得到具体 `int`。

D：✗ `sizeof(a)/sizeof(a[0])` 是行数；列数应是一行大小除以一个元素大小。

总结： 二维 `sizeof` 公式要看清“分子分母各代表哪一层”。

17 填空题详细解析

1. **答案：7。**

   `intarr[8]`；一共有 8 个元素，下标从 0 开始，依次为 0,1,2,3,4,5,6,7。

   易错：不要填 8；8 是元素个数，不是最后下标。

2. **答案：`sizeof(arr)/sizeof(arr[0])`。**

   分子是 6 个 `int` 总字节数，分母是一个 `int` 的字节数，相除得到 6。

   使用条件：这里的 `arr` 是当前作用域真实数组对象。

3. **答案：4。**

   因为 $n=5$，合法下标 0–4，所以右端是 $5-1=4$。

   易错：若填 5，第一次访问 `arr[5]` 就越界。

4. **答案：`n-i-1`。**

   第 i 轮开始前，最右边已有 i 个元素不必再参与；比较一对相邻元素又需要保证 `j+1` 合法，因此上界为 `n-i-1`。

   检查：第 0 轮是 `j<n-1`，刚好比较 $n-1$ 对。

5. **答案：`sizeof(a[0])/sizeof(a[0][0])`。**

   `a[0]` 是一整行，包含 7 个 `int`；用一行字节数除以一个元素字节数，得到列数 7。

   易错：`sizeof(a)/sizeof(a[0])` 求的是 4 行，不是 7 列。

18 基础解答题详细解析

第 1 题：寻找最大值

题意分析：遍历 5 个整数，维护当前最大值。

已知：`intarr[5]=\{12,7,19,3,10\};`

使用原则：初始最大值取数组第一个元素，然后从第二个元素开始比较。

```
int maxValue = arr[0];
for (int i = 1; i < 5; ++i) {
    if (arr[i] > maxValue) {
        maxValue = arr[i];
    }
}
```

逐步：初始 12；看到 7 不更新；看到 19 更新为 19；看到 3、10 均不更新。最终 $\boxed{19}$。

为什么不建议设 0：本题数据全为正数所以 0 恰好不影响结果，但如果数组全是负数，例如 {-5,-2,-9}，0 会错误地成为“最大值”。

同类题方法：找最小值只需把比较符号改成 `<`，并同样从 `arr[0]` 初始化。

第 2 题：原地逆置

知识定位：双指针 + 交换。

边界： start=0，end=4；循环只在 start<end 时进行。

```
int start = 0;
int end = 4;

while (start < end) {
    int temp = arr[start];
    arr[start] = arr[end];
    arr[end] = temp;
    ++start;
    --end;
}
```

第一轮： 交换 1 与 5，得到 $5, 2, 3, 4, 1$。

第二轮： 交换 2 与 4，得到 $5, 4, 3, 2, 1$。

此时 start=end=2，中间元素 3 不需要交换。最终 $\boxed{5, 4, 3, 2, 1}$。

结果检查： 逆置前后元素集合不变，只是顺序相反。

第 3 题：冒泡第一轮

初始数组：$[5, 1, 4, 2, 3]$。

比较 5 与 1： 交换 $\to [1, 5, 4, 2, 3]$。

比较 5 与 4： 交换 $\to [1, 4, 5, 2, 3]$。

比较 5 与 2： 交换 $\to [1, 4, 2, 5, 3]$。

比较 5 与 3： 交换 $\to [1, 4, 2, 3, 5]$。

所以第一轮结果为

$$\boxed{[1, 4, 2, 3, 5]}.$$

为什么最右端确定？因为最大值 5 一旦在扫描中出现，只要它右边还有更小元素，就会不断被交换向右；第一轮扫描到末尾后，再没有任何未检查位置在它右侧，因此它已经处于最终最大元素位置。

第 4 题：二维成绩逐行求和

知识定位： 外层遍历学生（行），内层遍历科目（列）。

```
for (int i = 0; i < 3; ++i) {
    int sum = 0;
    for (int j = 0; j < 3; ++j) {
        sum += score[i][j];
    }
    cout << sum << '\n';
}
```

第 0 行： $100 + 100 + 100 = 300$。

第 1 行： $90 + 50 + 100 = 240$。

第 2 行： $60 + 70 + 80 = 210$。

因此三名学生总分依次为

$$\boxed{300,\ 240,\ 210}.$$

同类题：如果要计算每门课的班级总分，只需“固定列、遍历行”，也就是把求和方向从按行改成按列。

19 高频易错点总表

错误	为什么错	规范写法 / 检查方式
arr[n] 当最后元素	下标从 0 开始	最后元素是 arr[n-1]
max=0 通用找最大值	全负数组会失败	max=arr[0]
inttemp[n]（n 运行时）	标准 C++ 原生数组边界要求常量表达式	用 constexpr 固定长度或 std::vector
逆置 end=n	第一次就越界	end=n-1
交换少一行赋值	两边值无法真正互换	临时变量三步交换
冒泡内层写 j<n	会访问 arr[j+1]，末尾越界	j<n-i-1
二维行列公式写反	分不清“总表/一行/一个格子”	先写出每个 sizeof 的语义再相除
函数参数里用数组长度公式	参数已调整成指针	显式传长度或用容器/模板

20 现代 C++ 的下一步：知道即可，不要求现在掌握

拓展联想：std::array 与 std::vector

本专题刻意使用课堂正在学习的 C 风格原生数组，因为它能帮助你理解连续内存、下标和指针基础。但在现代 C++ 工程实践中，固定长度数据常可优先考虑 std::array<T,N>，运行时长度数据常用 std::vector<T>；Microsoft Learn 也明确建议现代 C++ 优先使用这些标准库容器来获得更好的类型安全与接口。[R2]

当前阶段只需要知道：**原生数组是基础；标准容器是后续更安全、更工程化的工具。**

21 参考资料

R1. cppreference, *Array declaration (C++)*, 在线页面。核验数组声明、连续存储、多维数组与数组到指针转换。

R2. Microsoft Learn, *Arrays (C++)*, 在线页面。核验数组边界、初始化与现代 C++ 容器建议。

R3. cppreference, *sizeof operator (C++)*, 在线页面。核验 sizeof 的字节大小语义。

R4. Harvard University CS50, *Lecture 3: Algorithms / Bubble Sort*, 在线页面。核验冒泡排序过程与复杂度。

访问日期均为 2026-09-06。

学习主线：连续存储 → 下标 → 遍历 → 扫描/交换 → 双重循环 → 二维表格。数组的核心价值，是让一批同类型数据能够被程序系统地访问、比较、移动和统计。